

智能合约

陈润宇

对外经济贸易大学信息学院



内容提纲

- 智能合约概念
- 智能合约架构
- 智能合约运作机理
- 典型智能合约
- 开发智能合约

智能合约发展史

- 1994年美国密码学家尼克.萨博将智能合约定义为执行合约条款的计算机化交易协议。其设计初衷是在无须第三方可信权威的情况下，作为执行合约条款的计算机交易协议，嵌入某些由数字形式控制具有价值的物理实体，担任合约各方共同信任的代理，高效安全履行合约并创建多种智能资产。
- 自动贩卖机、销售点情报管理(Point of Sales, PoS)系统、电子数据交换(Electronic Data Interchange, EDI)系统都可看作是智能合约的雏形。因为缺乏能够支持可编程合约的数字系统和技术，很长一段时间内智能合约没有得到广泛的应用。
- 2008年比特币诞生后，人们认识到比特币的底层技术(即区块链)可以为智能合约提供可信的执行环境。以太坊重新使用了智能合约这一概念，在白皮书《以太坊:下一代智能合约和去中心化应用平台》中建立了一整套智能合约的规范与架构。
- 区块链的分布式账本技术不仅可以支持可编程合约，而且具有去中心化、不可篡改、过程透明可追踪等优点，适合于不受第三方控制按照预定的设定执行条款的智能合约。因此也可以说，智能合约是区块链技术的特性之一。
- 几乎所有类型的金融交易都可以被改造成在区块链上使用，包括股票、私募股权、众筹、债券和其他类型的金融衍生品如期货、期权等。区块链智能合约不仅可以被用来创建、确认、转移各种不同类型的数字资产，而且可以应用于多方参与的复杂业务应用场景，如电动车租赁、征信管理等。

智能合约概念

- 智能合约是各参与方共同制定的由计算机程序描述的协议，表现为一段代码，无须第三方干预，当一个预先设置的条件被触发时，智能合约执行相应的条款。
- 其中，①定义了选民类型，包含选民详细信息；②定义了提案类型，包含提案详细信息，目前只有voteCount；③将选民地址映射到选民详细信息；④定义了投票的各个阶段(0, 1, 2, 3)，状态初始化为Init 阶段。

```
pragma solidity >= 0.4.22 <= 0.6.0;  
  
contract BallotV1 {  
    struct Voter {  
        uint weight;  
        bool voted;  
        uint vote;  
    }
```

```
    struct Proposal {  
        uint voteCount;  
    }  
  
    address chairperson;  
    mapping(address => Voter) voters;  
    Proposal[] proposals;  
    enum Phase {Init, Regs, Vote, Done}  
    Phase public state = Phase.Init;  
}
```

智能合约的功能

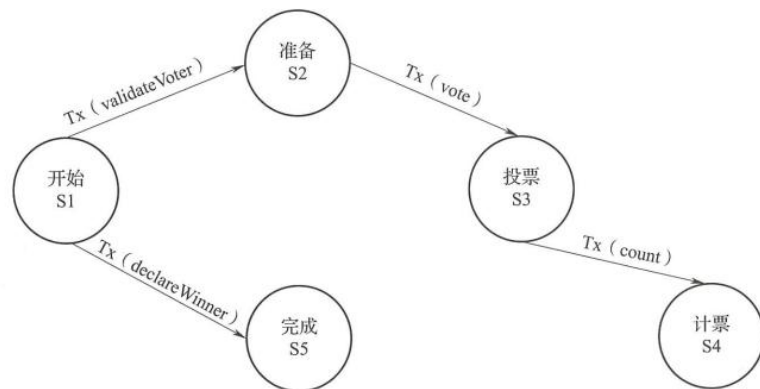


图 5-1 投票智能合约

■在支持智能合约的区块链中，交易嵌入了由智能合约实现的功能。图5-1 中展示了投票智能合约。validateVoter(), vote(), count()和 declareWinner()等方法的调用会触发记录在区块链上的交易[Tx(validateVoter), Tx(vote), Tx(count), Tx(declare-Winner)等]。除了简单的加密货币转账功能，在区块链上部署任意逻辑的能力大大增强了区块链的适用性。

■智能合约是区块链应用程序的核心，包括以下功能

- 它代表用于验证和确认特定应用程序条件的业务逻辑层。
- 它允许定义区块链上的操作规则。它促进了分布式网络中资产转移策略的实施。它嵌入了可以由参与者账户或其他智能合约账户的消息或函数调用的功能。这些消息及其输入参数，以及其他元数据(如发件人的地址和时间戳)，是交易被记录在区块链的证明。
- 它可以作为区块链应用程序的软件中介程序。
- 它通过定义功能参数为区块链增加了可编程性和智能性。

智能合约与传统合约的比较

- 智能合约和传统合约有着明显差异
- 智能合约分为广义智能合约和狭义智能合约。
 - 广义的智能合约是指运行在区块链上的计算机程序，适用范围较广。
 - 狭义的智能合约运行在区块链基础架构上，其基于约定规则，由事件驱动、具有状态、能够保存是账本上资产，利用程序代码来封装和验证复杂交易行为，实现信息交换、价值转移和资产管理，是可自动执行的计算机程序。
 - 使用区块链网络，传统合约可以转化为可执行程序，也就是智能合约，从而带来较多应用可能性。智能合约的执行方式除了不用考虑第三方的干预和干扰，在工作效率上也比人工执行更高效。

表 5-1

智能合约与传统合约的比较

比较内容	智能合约	传统合约
触发条件	自动判断	人工判断
适应场景	适合客观性的请求	适合主观性的请求
成本	低成本	高成本
执行模式	事前预防	事后执行
违约成本	依赖于抵押品或保证金	高度依赖于法律的执行
适用范围	可以是全球的	受限于具体辖区

智能合约的优缺点

■ 优点

■ 可信性

- 智能合约的所有条款和执行过程是提前制定好的，并由计算机绝对执行。因此所有执行的结果都是准确无误的，不会出现不可预料的结果，因此是可信的。

■ 执行无须第三方

- 智能合约不需要中心化的权威来仲裁合约是否按规定执行，合约的监督和仲裁都由计算机来完成。在一个区块链网络中由共识机制来判断合约是否按规定执行，监督方式通常由PoW或PoS技术实现。由于智能合约的数字化特点，数据被存储在区块链中，使用加密代码强制执行协议，保证交易可追踪和不可逆转。

■ 高效实时更新

- 由于智能合约的执行不需要人为的第三方权威或中心化代理服务的参与，它能够在任何时候响应用户的请求，大大提升了交易进行的效率。

■ 低成本

- 合约制定人在合约建立之初就确定合约的各个细节，就能充分利用智能合约去除人为干预的特点，大大减少合约履行、裁决和强制执行所产生的人力成本。

智能合约的优缺点

■ 存在的问题

■ 不可撤销性

- 智能合约自动履行合约内容，但在现实生活中，合同可能会因为一些不可抗力、违法等原因解除。智能合约一旦触发就会自动履行，不可撤销（可以提前共识修改）。

■ 法律效力

- 智能合约的起草是需要通过第三方计算机程序员实现的。如果合约出现了因程序员责任导致的问题，如何由错误的程序追究相应的责任是需要解决的问题。在法律管辖权问题上，智能合约作为一种新兴合约方式，哪些法院可以受理诉讼、现有的法律条款应该如何修改等问题都是亟待解决的问题。

■ 安全漏洞

- 智能合约的漏洞分为交易顺序依赖漏洞、时间戳依赖漏洞、处理异常漏洞和可重入缺陷漏洞。攻击者可通过更改交易顺序、修改时间戳、调用可重入函数、触发处理异常等影响智能合约执行结果或窃取资金。

■ 恶意合约

- 区块链及智能合约的去中心化、匿名性同样可能助长恶意合约的产生。违法者可通过发布恶意的智能合约对区块链系统和用户发起攻击，也可利用合约实现匿名的犯罪交易，导致机密信息的泄露、密钥窃取或各种真实世界的犯罪行为。

智能合约安全性问题

安全案件	威胁层面
“DAO”, “KotET” (2016)	虚拟机层面、高级语言层面
“Parity Multisig” (2017)	虚拟机层面
“BEC/SMT”整数溢出 (2018)	高级语言层面
“EODPlay” 随机数攻击 (2019)	区块链层面
“Uniswap”重入攻击 (2020)	虚拟机层面
Poly Network 黑客案 (2021)	虚拟机层面
虫洞攻击 (2022)	虚拟机层面

- 2016年，The Dao开启众筹，仅仅一个月时间就募集到价值1.5亿美元的以太币。然而，黑客发现了The Dao智能合约的漏洞，并对其发动了攻击，大量的以太币被黑客盗走。由于智能合约一旦部署无法修改，团队也无能为力，只能看着巨额资产逐渐流失。随后，以太坊创始人提出回滚交易的方案，但由于社区部分用户并不赞同回滚，从而导致了以太坊的硬分叉事件。
- 借贷协议bZx也曾多次因为智能合约代码漏洞遭到黑客攻击。2020年2月15日，bZx团队在官方电报群上发出公告，称有黑客对bZx协议进行了漏洞攻击，导致价值35万美元的ETH被盗。同年9月14日，DeFi借贷协议bZx再次遭到攻击，这次攻击共造成了大约800万美元的损失。
- 智能合约是管理区块链平台上的数字资产的关键组成部分，其安全性关乎区块链平台上用户数字资产的安全性，智能合约攻击事件的频繁发生对受害者合约带来巨大的经济损失。
- 智能合约本质上是部署和运行在区块链上的程序，在没有标准的合约模版或编写规范的情况下很难写出最佳实践的代码，一些逻辑不严谨的代码也会造成智能合约业务逻辑存在安全漏洞。

智能合约的安全性问题

- 智能合约的漏洞分为交易顺序依赖漏洞、时间戳依赖漏洞、处理异常漏洞和可重入缺陷漏洞。攻击者可通过更改交易顺序、修改时间戳、调用可重入函数、触发处理异常等影响智能合约执行结果或窃取资金。
 - 依赖性漏洞是由于智能合约的执行正确与否与以太坊的状态有关。当一个新的区块含有两笔交易时，交易的先后顺序可能会引起以太坊的最终状态不同。交易的顺序取决于矿工，导致智能合约的执行依赖于矿工的合法操作。
 - 时间戳依赖漏洞是由于某些智能合约是根据区块中的时间戳所执行的，而时间戳是由矿工根据自身的时间所设置的，若时间被攻击者所修改，可能会导致产生一定风险。
 - 在不同的智能合约相互调用时可能出现处理异常漏洞，若被调用的合约产生错误返回值却没有被正确验证时，可能会遭受到攻击。
 - 若一个函数在执行完成前被调用了数次，导致发生意料不到的行为时，可重入缺陷漏洞就可能出现。可重入缺陷漏洞是指攻击者可以利用调用了智能合约而状态未改变的中间状态对合约进行反复的调用。
- 设计一个安全的智能合约的难点在于所有网络参与者都可能出于自身利益攻击或欺骗智能合约，设计者必须预见一切可能的恶意行为，并设置应对措施。经 Oyente检查发现，在19366个以太坊智能合约中，有8833个存在上述至少一种安全漏洞。此外，无可信数据源和待优化智能合约也将带来一定经济损失，攻击者可通过向合约输入虚假数据获取经济效益，用户则需为无用代码额外付费。

智能合约的安全代码要求

- 与其他区块链安全问题相比，智能合约安全性问题往往会引发较大的经济损失且更加不可控。为了减少安全事件的发生，一方面，可以从智能合约运行的环境入手，而另一方面，由于智能合约通常具有发布即生效、不可更改等特点，需要在其上线前进行全面深入的安全验证，在开发阶段就尽可能对安全隐患有所考虑并进行规避，基于一定的安全原则进行合约代码的编写，进而减少智能合约自身的安全漏洞。基于现有的智能合约已经出现的安全问题案例，以以太坊的智能合约开发体系Solidity为例，智能合约开发中需要注意的重点内容可以包含如下几点。
 - 严格控制表征权限的变量和表示。访问控制权限是安全领域的防护重点之一。在智能合约中，针对权限，主要考虑是否为合约所有者、是否满足用户添加的权限要求、在合约内部还是外部等。其实现依赖函数修饰符、判断语句等，通常会有变量或标识来表征。因此，必须对用于表征权限的变量和表示进行严格的控制，即这些敏感变量也应该通过函数修饰符进行权限控制，从而保证权限闭环，让攻击者无机可乘。
 - 尽量避免外部调用。调用不受信任的外部合约可能会引发一系列意外的风险和错误，外部调用可能在其合约和它所依赖的其他合约内执行恶意代码。因此在实现业务逻辑时应避免直接调用不受信任的外部合约，多花时间重复实现其他合约已经实现的业务逻辑，宁肯重复“造轮子”，以规避合约引入恶意代码攻击的风险。
 - 在智能合约的开发中，应当始终保持简洁。复杂的智能合约往往存在更多的安全隐患，因此对于智能合约的开发，清晰明了往往比性能提升更为重要。智能合约开发应确保逻辑简洁，坚持通过内部模块化和功能调用的编码风格实现合约逻辑，同时尽量使用已被广泛验证过的合约和工具。

智能合约应用场景

- 智能合约的应用（非区块链）学校里有吗？
 - 无人自动售卖机，取出商品后自动扣款，完成一次商品交易。
- 电子合同中的应用
 - 当多方用户进行合同签署时，电子合同是将传统合同签署流程转移到互联网上，传统合同的签署过程需要双方在纸质合同上签署盖章然后邮寄，电子合同平台则通过用户的证书授权中心(Certificate Authority, CA)机构颁发的证书对双方共同签署的合同给予签章。
 - 相对于传统纸质合同需要双方在同一份合同上签字，不论是面对面签署产生的交通成本，还是将一方签署完的合同寄往另一方所产生的邮寄成本，还有这期间所耗费的时间成本，电子合同都可以很好地解决这些问题。
 - 在安全性和可靠性方面，电子合同还是比较依赖第三方机构，也就是服务提供商的信任背书。电子合同平台通过作为中立的第三方角色来证明自己不会篡改合同，但是从技术上来说，电子合同平台是有可能篡改合同内容的。因此，大多数人对电子合同的安全性和信任度不高，受到诸如主客观、经济成本、适用范围、执行力度和执行时间等因素的影响，这一定程度上阻碍了电子合同在数字经济中的价值实现。

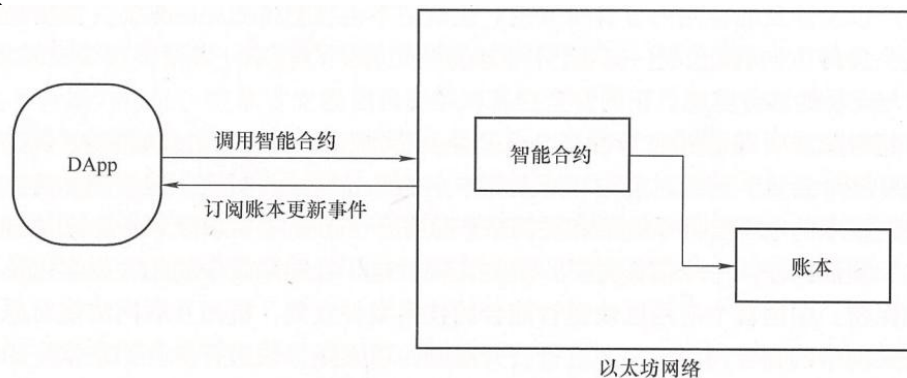
智能合约应用场景

- 电子合同中的应用（智能合约与电子合同结合）
- 区块链技术利用密码学算法、独特的数据结构以及共识算法，使电子合同签署的多方用户在交易记录上的一致性、签字后合同的存储和管理的可靠性，以及后期防止电子合同被篡改的安全性等问题都由机器算法来进行处理，结合其无须信任的特性，可以作为完美的中立第三方角色。电子合同的签署会被记入一个共享账本上，该账本用以存储合同，并且是多方用户共同维护，不允许用户篡改信息，无法抵赖，同时也不会出现遗失的情况。
- 对于电子合同的文本以及要素进行加密储存处理，能防止合同参与者的敏感信息也外泄，只有参与者和提前指定的中介或监管机构才能够对其解密和查看内容，从数据级别来对电子合同以及相关人员的敏感信息展开保护。机器会根据提前设置好的智能合约代码来执行而不是通过第三方机构，成为电子合同安全、可靠及合法的有力保障。
- 按照《电子签名法》和《电子合同在线订立流程规范》，需要中立第三方服务提供商的订立系统和存储服务，而保密原则的实施则只能靠与第三方签署的保密协定来保证。可以通过与公证处组成联盟链来实现合同存证信息的共享。合同存证信息一旦写入区块链，即会自动同步到公证处数据节点，区块链每个节点保存了全量的电子合同存证信息，保证任意一个节点都无法随意篡改合同内容。如果合同发生纠纷，也可以在公证处查证合同的真实性，从技术上保证了电子合同涉及的任何一方都不可能篡改合同。

智能合约应用场景

■ DApp

- 分布式应用(Decentralized Application, DApp)是可运行在分布式网络即区块链网络上的应用程序。DApp 与 App 的客户端区别不大,但是服务端则不同,DApp 通常会部分或者全部部署在区块链网络上。
- DApp 具有四个基本的特点:开源、拥有Token 激励机制、去中介化的共识机制和无单点故障缺陷。区块链上的用户数据通常是用加密方式存储,数据的所有权归属用户,而非 DApp 的开发者;DApp 中消耗的资源由通证经济模型予以补偿或激励;DApp 的后端程序是部署在区块链上的智能合约,智能合约是一组预定义的业务规则,具备确定性(Deterministic)执行的特征,能有效降低信任成本。
- 如图 是 DApp 工作流程,当 DApp 发送的事件以及交易请求被智能合约正确接收到后,就会触动预编写完成的代码逻辑,引发区块链中的账本项目完成状态操作。DApp将调出智能合约所提供的接口,进而执行相应的业务逻辑,利用封装智能合约和账本进行直接交互的指令完成对上一层级业务逻辑工作的支持



智能合约应用场景

■ DApp应用案例

- 基于以太坊开发平台的游戏类 DApp 以太猫 CryptoKitties 于2017 年发布上线，其内容是一款虚拟养猫游戏，用户可买卖并繁殖不同品种的电子宠物小猫。在这个游戏当中，用户可以收藏、交易和繁殖以太猫，有别于比特币这类加密货币，以太猫更像加密收藏品，这意味着用户的以太猫所有权始终属于用户，所有权由智能合约确定，而智能合约是无法关停的，这点也是它区别于 Pokemon 的地方，因为一旦Pokemon 背后的公司倒闭，用户所拥有的宠物也随之消失。而且作为收藏品，以太猫的市场价格是由市场需求、本身的稀缺性和用户的报价决定的。以太猫游戏中的每只猫都是独一无二的，每只小猫都有256 组基因，不同的基因组合会让小猫的背景颜色、长相和条纹等都有差异，甚至还有隐性基因的设计。用户可为自己的小猫命名，并通过各种营销手法，让自己小猫的更好销售，买卖猫咪成了以太猫游戏的一大特色。由于受到大量数字加密货币爱好者的热捧，游戏上线之后就传播迅速，曾一度造成以太坊网络交易拥堵。
- 基于区块链技术的DApp 还主要出于实验阶段，并未有大规模实际应用价值的 DApp 获得成功，主要呈现出的倾向是娱乐化、抽奖以及游戏等相关的带有法律风险的 DApp，其他具有现实意义的 DApp 还具有非常大的开发和创造空间。DApp 产品的具体落地实施还是要考虑很多的相关因素，主要问题是以太坊平台处理效率低，以太坊的机制以及运行效率目前还很难支持一个庞大的分布式商业应用生态。商业级 DApp 的落地需要基于一个智能合约速度更快、扩展性更强、安全性更高的基础设施。

智能合约架构

■ 智能合约的抽象模型

- 一段代码(智能合约)被部署到共享的、复制的分布式账本中，可以维护自身的状态，控制自身的资产并回应接收到的外部信息或者资产。
- 公共记账本是一种状态转换系统，记录任何账户所持有货币的所有权状态以及预先定义的“状态转换函数”。当该系统接收到一个(可以由交易或者可信外部事件引发)含有状态改变的事务时，它将根据状态转换函数输出一个新的状态，并将该输出状态(以一种所有人都信任的方式)写入公共记账本。这一过程可以往复进行。

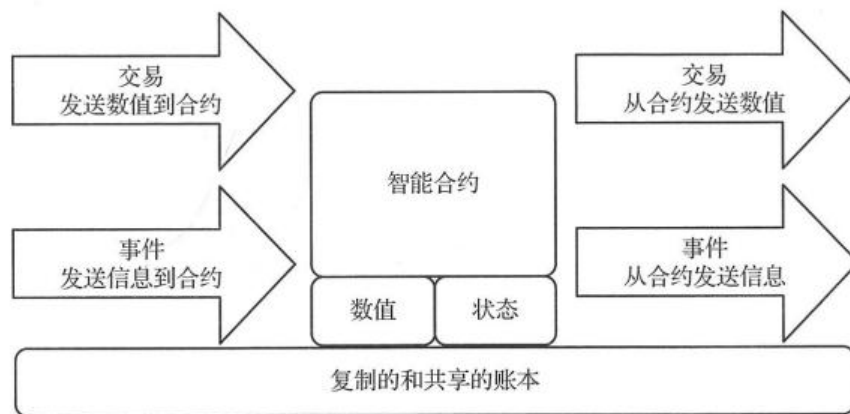


图 5-2 区块链智能合约抽象模型

智能合约架构

■ 智能合约通用架构

- 狭义的智能合约可看作是运行在区块链上的计算机程序，作为计算机程序，智能合约的开发、部署和调用将涉及包括编程语言、集成开发环境(Integrated Development Environment, IDE)、开发框架、客户端等多种专用开发工具。
- 区块链智能合约的通用架构包括编程语言集成开发环境、部署环境和运行环境。



图 5-3 区块链智能合约通用架构：以以太坊为例

智能合约运作机理

■ 智能合约运行原理

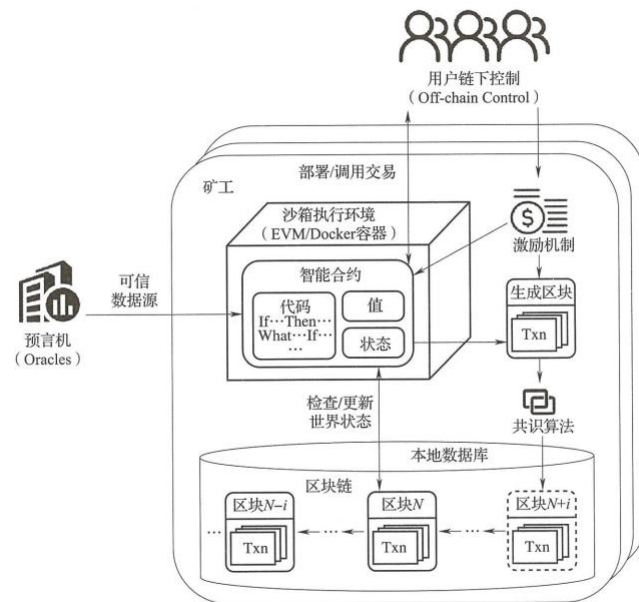


图 5-4 智能合约的运行机制①

- 世界状态：可以被视为随着交易的执行而持续更新的全局状态，其保存了区块链所有的账户信息。如果想知道某一账户的余额，或者某智能合约当前的状态，就需要通过查询世界状态树来获取该账户的具体状态信息。
- 如图所示，智能合约一般具有值和状态两个属性，代码中用条件判断(If_Then)和假设分析(What-If)语句预置了合约条款的相应触发场景和响应规则，智能合约经多方共同协定、各自签署后随用户发起的交易提交，经点对点等网络传播、矿工验证后存储在区块链特定区块中，用户得到返回的合约地址及合约接口等信息后即可通过发起交易来调用合约。

智能合约运作机理

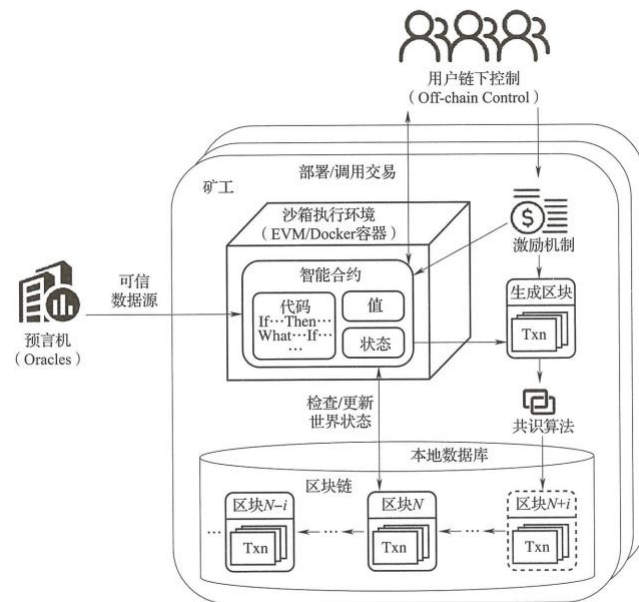


图 5-4 智能合约的运行机制①

■ 智能合约运行原理

- 基于系统预设的激励机制，矿工将贡献自身算力来验证交易，矿工收到合约创建或调用交易后在本地沙箱执行环境(如以太坊虚拟机)中创建合约或执行合约代码，合约代码根据可信外部数据源(也称为预言机，Oracles)和世界状态的检查信息自动判断当前所处场景是否满足合约触发条件以严格执行响应规则并更新世界状态。
- 交易验证有效后被打包进新的数据区块，新区块经共识算法认证后链接到区块链主链，运行合约产生的所有更新得以生效。由于区块链种类及运行机制的差异，不同平台上智能合约的运行机制也有所不同，以太坊和超级账本是目前应用广泛的两种智能合约开发平台，它们的智能合约运行机制最具有代表性。
- 基于区块链的智能合约包括事务处理和保存的机制，以及一个完备的状态机，用于接受和处理各种智能合约，并且事务的保存和状态处理都在区块链上完成。
- 事务主要包含需要发送的数据，而事件则是对这些数据的描述信息。事务及事件信息传入智能合约后，合约资源集中的资源状态会被更新，进而触发智能合约进行状态机判断。如果自动状态机中某个或某几个动作的触发条件满足，则由状态机根据预设信息选择合约动作自动执行。

智能合约运作机理

■ 基于区块链的智能合约构建及执行步骤

- 步骤1：多方用户共同参与制定一份智能合约。
 - (1)首先用户必须先注册成为区块链的用户，区块链返回给用户一对公钥和私钥。公钥作为用户在区块链上的账户地址，私钥作为操作该账户的唯一钥匙。
 - (2)两个以及两个以上的用户根据需要，共同商定了一份承诺，承诺中包含了双方的权利和义务。这些权利和义务以电子化的方式变成计算机程序语言。参与者分别用各自私钥进行签名以确保合约的有效性。
 - (3)签名后的智能合约将会根据其中的承诺内容传入区块链网络中。
- 步骤2：合约通过点对点网络扩散并存入区块链。
 - (1)合约通过点对点网络的方式在区块链全网中扩散，每个节点都会收到一份。区块链中的验证节点会将收到的合约先保存到内存中，等待新一轮的共识时间，触发对该份合约的共识和处理。

智能合约运作机理

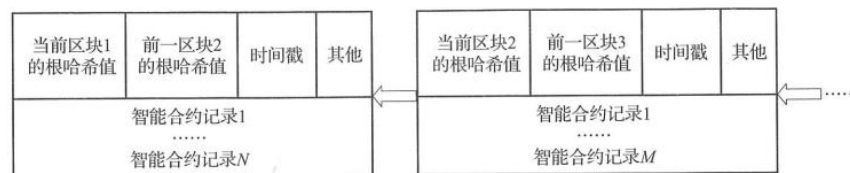


图 5-5 合约区块链示意图

■ 基于区块链的智能合约构建及执行步骤

■ 步骤2：合约通过点对点网络扩散并存入区块链。

- (2)在共识阶段，验证节点会将最近一段时间内保存的所有合约打包成一个合约集(Set)，并算出这个合约集合的哈希值，最后将这个合约集合的哈希值组装成一个区块结构，扩散到全网，其他验证节点收到这个区块结构后，会将其包含的合约集合的哈希值取出来，与自身保存的合约集合进行比较，同时发送一份自身认可的合约集合给其他的验证节点。通过多轮的发送和比较，所有的验证节点最终在规定的时间内对最新的合约集合达成一致。
- (3)最新达成的合约集会以区块的形式扩散到全网，如图5-5所示，每个区块包含以下信息：当前区块的哈希值、前一区块的哈希值、达成共识时的时间戳以及其他描述信息；同时区块链最重要的信息是带有一组已经达成共识的合约集。收到合约集的节点，都会对每条合约进行验证，检查合约参与者的私钥签名是否与账户匹配，如果匹配，则验证通过。验证通过的合约会最终被写入区块链中。

智能合约运作机理

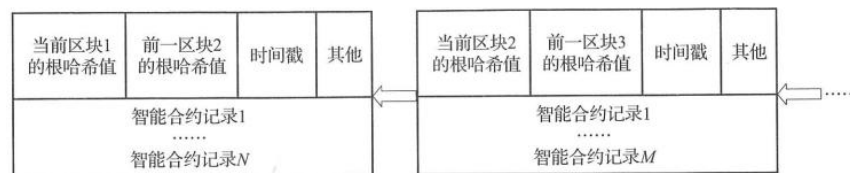


图 5-5 合约区块链示意图

■ 基于区块链的智能合约构建及执行步骤

■ 步骤3：区块链构建的智能合约自动执行。

- (1)智能合约会定期检查自动机状态，逐条遍历每个合约内包含的状态机、事务以及触发条件，将条件满足的事务推送到待验证的队列中，等待共识阶段的启动，未满足触发条件的事务将继续存放在区块链上。
- (2)进入最新轮验证的事务，会扩散到每一个验证节点，与普通区块链交易或事务一样，验证节点首先进行签名验证，确保事务的有效性。验证通过的事务会进入待共识集合，等大多数验证节点达成共识后，事务会成功执行并通知用户。
- (3)事务执行成功后，智能合约自带的状态机会判断所属合约的状态，当合约包括的所有事务都顺序执行完后，状态机会将合约的状态标记为完成，并从最新的区块中移除该合约。反之将标记为进行中，继续保存在最新的区块中等待下一轮处理，直到处理完毕。整个事务和状态的处理都由区块链底层内置的智能合约系统自动完成，全程透明、不可篡改。

运行合约代码

- 现有区块链系统中，智能合约的实现技术可以按照智能合约运行的环境进行划分，具体可分为3类：嵌入式运行、基于虚拟机运行和容器式运行。
- 表5-3列举了现有的主流区块链系统及其智能合约的应用类型、运行环境和编程语言。
 - 比特币的智能合约脚本为脚本型智能合约，仅包含指令和数据，只需完成有限的交易逻辑。
 - 图灵完备型智能合约可以实现各种复杂业务功能，以太坊、超级账本等。
 - 防止复杂合约可能存在安全漏洞，验证合约型语言采用非图灵完备设计，不支持循环和递归

表 5-3 不同区块链系统智能合约运行环境和语言比较

区块链系统	应用类型	智能合约运行环境	智能合约语言
以太坊	通用应用	以太坊虚拟机	Solidity, Serpent, Mutan
超级账本 Fabric	通用应用	容器	Golang, Java
比特币	加密货币	嵌入式运行	Golang, C++
Zcash	加密货币	嵌入式运行	C++
Quorum	通用应用	以太坊虚拟机	Golang
Parity	通用应用	以太坊虚拟机	Solidity, Serpent, Mutan
Litecoin	加密货币	嵌入式运行	Golang, C++
Corda	数字资产	Java 虚拟机	Kotlin, Java
Sawtooth	通用应用	嵌入式运行	Python

运行合约代码

- 以以太坊开发平台为例，智能合约运行机制主要包含以下阶段。
 - 生成代码:智能合约一般具有值和状态两个属性，代码中用If-Then 和What-If 语句预置了合约条款的相应触发场景和响应规则，在合约各方面内容都达成一致的基础上，评估确定该合同是否可以通过智能合约实现，即“可编程”，然后由程序员利用合适的开发语言将以自然语言描述的合同内容翻译为可执行的机器语言。
 - 编译:利用开发语言编写的智能合约代码一般不能直接在区块链上运行，而需要在特定的环境(以太坊为EVM，超级账本为 Docker 容器)中执行，所以在将合约文件上传到区块链之前需要利用编译器对原代码进行编译，生成符合环境运行要求的字节码。
 - 提交:智能合约的提交和调用是通过交易完成的，当用户以交易形式发起提交合约文件后，通过点对点网络进行全网广播，各节点在进行验证后将合约文件存储在区块中。
 - 确认:被验证后的有效交易被打包进新区块，通过共识机制达成一致后，新区块添加到区块链的主链。根据交易生成智能合约的账户地址，之后可以利用该账户地址通过发起交易调用合约，节点对经验证有效的交易进行处理，被调用的合约在环境中执行。
 - 智能合约的生命周期根据其运行机制可概括为协商、开发、部署、运维、学习和自毁6个阶段，其中开发阶段包括合约上链前的合约测试，学习阶段包括智能合约的运行反馈与合约更新等。

典型智能合约

表 5-4

以太坊和超级账本智能合约的比较

智能合约特性	以太坊平台	超级账本 Fabric 平台
执行环境	以太坊虚拟机	Docker
编写语言	Solidity, Serpent, Mutan	Golang, Java
合约部署	作为交易广播到所有节点, 通过矿工挖矿完成部署	直接在所有节点部署
合约升级	无法升级	V1.0 版本可以升级
合约间调用	可调用	许可后可调用
合约终止方式	计步计价, 引入燃料消耗, 对每一个执行命令都消耗燃料, 燃料消耗完毕强行终止合约	计时, 以运行时间为标准判定程序是否进入无限循环, 超时后强行终止合约
库函数	少量对整数、字符串、JSON 等封装的库	Golang 库
加密货币	内置加密货币以太币, 可以利用合约交易加密货币或者创建通证	无内置加密货币, 但可利用链码创建通证

- 以太坊定位为完全独立于任何特定应用领域的通用平台, 具有对应的账户和代币功能, 允许智能合约作为特殊的账户部署在区块链节点上, 应用程序通过应用程序接口调用节点上的智能合约来产生交易, 变更区块状态。
- 基于以太坊虚拟机的智能合约平台, 利用以太坊虚拟机, 可以运行具有任何交易方式、任何资产类型、任何权限分配等级的去中心化应用, 这使得基于EVM的智能合约相比于脚本智能合约有更强大的普适性, 在数字资产、股权、征信、物联网、医疗等许多领域都能发挥巨大作用。
- 超级账本Fabric为模块化和可扩展的架构, 不具有自身的代币, 共识机制采用实用拜占庭容错算法而非以太坊的工作量证明, 具有较高的共识效率, 而且共识服务从背书节点分离, 独立形成可插拔模块, 可扩展性强。其主要作为联盟链, 面向银行、医疗保健和供应链等行业。
- 基于 Docker 容器的超级账本Fabric 平台同样可以运行具有通用功能的智能合约代码, 而且因其引入了多通道设计, 使得不同通道互连节点之间的数据被隔离, 相比以太坊虚拟机平台具有更好的隐私性, 适用于数据敏感型商业应用。

典型智能合约：以太坊智能合约

- 以太坊虚拟机是一个完全独立的沙箱，合约代码在内部运行且对外隔离。
 - 以太坊虚拟机被部署在执行智能合约操作码的各个节点之上，负责对智能合约进行指令解码，并按照堆栈完全顺序执行代码。
 - 其结构不同于标准的冯诺依曼模型，程序代码并非保存在通用内存和永久存储，而是被置于特殊的交互式虚拟ROM中。虚拟机提供简单栈式结构，为了与Keccak256 哈希算法和椭圆曲线算法相匹配，栈的元素大小被设计为256 位。
 - 以太坊虚拟机本身运行一个状态函数，也称状态机，用于持续监测状态的变化。当新的进程触发时，以太坊虚拟机运行代码并将一定数据写入内存或永久存储，每一个新状态都基于上一个状态改变。
- 为了防止区块链网络资源滥用或由图灵完备引起的无限循环故障，任何可编程的计算都受计费限制。该计费机制以Gas为单位，创建智能合约、调用消息、访问账户存储的数据，并且虚拟机上的运行操作都对应一定的 Gas 计费标准。
- Gas 计费的引入为智能合约的运行提供了机制上的安全保障，一旦 Gas 超过计费限制 (GasLimit)，整个交易将会被回滚，以保证数据的完整性和安全性。

典型智能合约：以太坊智能合约

■ 以太坊合约的创建与运行

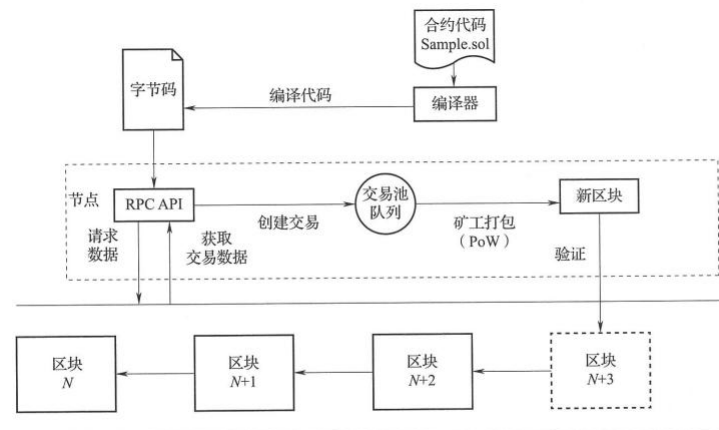
- 以太坊系统中，创建合约可看作为一种特殊的交易过程，合约创建函数利用一系列固定参数实现新合约的创建，并产生一组新的状态，过程如下：

$$(\sigma', g', A) \equiv \Lambda(\sigma, s, o, g, p, v, i, e)$$

- 其中， o 为系统状态， s 为交易发送者， O 为交易源账户主体， g 为可用燃料， P 为燃料价格， v 为账户金额， i 为初始化以太坊虚拟机代码， e 为创建合约栈的深度， o' 为系统新状态， g' 为剩余燃料， A 为子状态。最终，通过执行初始化EVM 代码，创建新的合约账户，产生账户地址、存储空间以及账户的主体代码。
- 该过程中，除去发生交易所消耗的燃料，代码创建的燃料消耗量与所创建合约的代码量成正比。然而，一旦燃料剩余量小于代码创建所需燃料，则会产生燃料异常(Out of Gas, OOG)，并且燃料剩余量将被置为零，也不再创建新的合约。合约运行模型则描述了在接收一系列字节码和环境数据元组之后，系统状态的转变方式。在实际运行中，该模型由全系统状态和虚拟机状态的迭代过程构成。迭代器不停运行迭代函数，直到状态异常或产生正常结果而暂停。

典型智能合约：以太坊智能合约

■ 以太坊合约的创建与运行：部署流程



- (1)编写智能合约代码，形成合约代码文件(如 Sample.sol);
- (2)通过智能合约编译器对代码文件进行编译，将其转换成可以在以太坊虚拟机执行的字节码;
- (3)向区块链节点RPC API发送创建交易(部署合约)请求，交易被验证合法后，识别为合约创建交易，检查输入数据，进入交易池;
- (4)矿工打包该交易，生成新的区块，并广播到点对点对等网络;
- (5)节点接收到区块后对交易进行验证和处理，为合约创建以太坊虚拟机环境，生成智能合约账户地址，并将区块入链;
- (6)API获取智能合约创建交易的收据，得到智能合约账户地址，部署完成。
- 智能合约调用流程与部署流程类似，也是通过 RPC API 创建交易，并由验证节点对交易进行处理，调用以太坊虚拟机实例，变更状态。

典型智能合约：超级账本智能合约

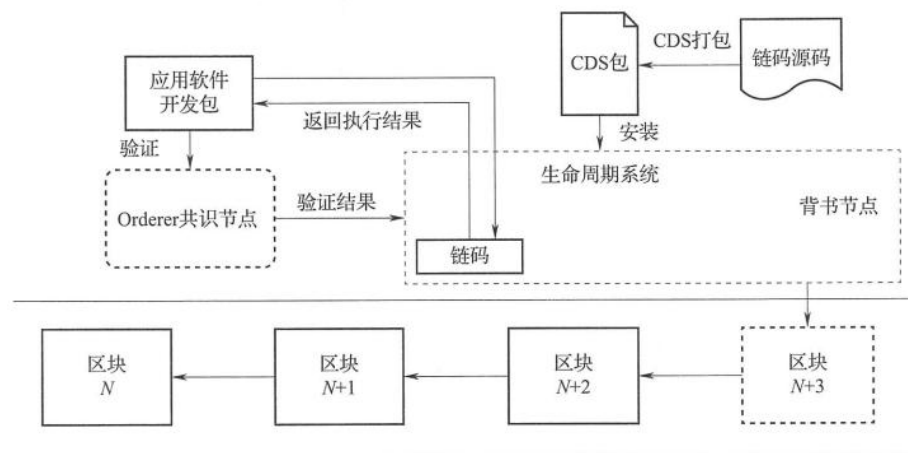
- 超级账本Fabric 是最早由IBM牵头发起的致力于打造区块链技术开源规范和标准的联盟链，2015年起成为开源项目并移交给Linux 基金会维护。
- 超级账本只允许获得许可的相关商业组织参与、共享和维护，由于这些商业组织之间本身就有一定的信任基础，超级账本被认为并非完全去中心化。超级账本使用模块化的体系结构，开发者可按需求在平台上自由组合可插拔的会员服务、共识算法、加密算法等组件组成目标网络及应用。
- 链码是超级账本Fabric 中的智能合约，开发者利用链码与超级账本交互以开发业务、定义资产和管理去中心化应用。联盟链中每个组织成员都拥有和维护代表该组织利益的一个或多个对等节点，联盟链由多个组织的对等节点共同构成。
- 对等节点是链码及分布式账本的宿主，可在 Docker 容器中运行链码，实现对分布式账本上键值对或其他状态数据库的读/写操作，从而更新和维护账本。

典型智能合约：超级账本智能合约

- 超级账本Fabric 的运行过程包含三个阶段
 - 1.提议(Proposal)
 - 应用程序创建一个包含账本更新的交易提议，并将该提议发送给链码中背书策略指定的背书节点集合(Endorsing Peers Set)做签名背书。每个背书节点独立地执行链码并生成各自的交易提议响应后，将响应值、读/写集合和签名等返回给应用程序。当应用程序收集到足够数量的背书节点响应后，提议阶段结束。
 - 2.打包(Packaging)
 - 应用程序验证背书节点的响应值、读/写集合和签名等，确认所收到的交易提议响应一致后，将交易提交给排序服务节点(Orderer)。排序节点对收到的众多交易进行排序并分批打包成数据区块后将数据区块广播给所有与之相连接的对等节点。
 - 3.验证(Validation)
 - 与排序节点相连接的对等节点逐一验证数据区块中的交易，确保交易严格依照事先确定的背书策略由所有对应的组织签名背书。验证通过后，所有对等节点将新的数据区块添加至当前区块链的末端更新账本。需要注意的是，此阶段不需要运行链码，链码仅在提议阶段运行。链码、交易通过链码执行、超级账本嵌在交易中，所有验证节点在确认交易时必须执行。执行环境是一个定制化、安全的沙箱，链码中需要持久化的状态，可以存储在世界状态中。

典型智能合约：超级账本智能合约

■ 链码在超级账本安装部署及运行流程



- (1)链码源码以链码部署规范(Chaincode Deployment Spec, CDS)签名生成CDS包;
- (2)通过生命周期系统链码, 将链码部署规范包安装在同一通道内的背书节点上, 生成运行在节点上的链码;
- (3)应用程序通过应用软件开发包发送请求到背书节点;
- (4)节点通过链码执行交易并将执行结果返回给应用程序;
- (5)应用程序收集结果, 将结果发送给共识排序服务节点(Orderer);
- (6)共识节点执行共识过程并生成区块验证结果;
- (7)背书节点各自验证交易并提交到超级账本区块中。

智能合约开发步骤

■ 步骤一：理解合约类图

- 合约类图是用来表示智能合约中的类、接口、协作以及它们之间的静态结构和关系的一种静态模型。
- 它主要用于描述智能合约的结构化设计，帮助人们理解智能合约。它是智能合约分析与设计阶段的重要产物，也是智能合约编码与测试的重要模型依据。
- 合约类图一般由工程师设计完成，理解合约类图有助于了解智能合约设计方式和智能合约的具体编码实现。在阅读合约类图的时候，要注意以下事项：
 - 理解数据类型的表示方法，以及各种数据类型之间的关系。
 - 理解各种方法之间的联系及其方法相关的业务逻辑，以及相应的约束规则。

■ 步骤二：开发智能合约

- 理解合约设计方案以后，就可以着手开发智能合约。开发智能合约需要基于一定的开发原则，针对不同区块链系统采取不同的开发方法，区分智能合约的查询与调用操作，实现智能合约的异常处理机制，并合理使用智能合约事件以便实现历史追溯事务。

智能合约开发步骤

■ 步骤二：开发智能合约

■ 智能合约开发原则

- 智能合约作为一种运行在分布式系统中的新型程序逻辑，与目前主流的应用系统开发存在较大的区别，智能合约程序通常运行在受限的安全沙箱内，并且每一次对智能合约的调用都会在所有区块链节点上执行。开发实现应遵循以下原则。
- 1. 资源限制原则：当有新的合约调用发生时，所有区块链节点都会验证并执行智能合约，合约代码会在所有节点的本地安全沙箱内冗余执行，应尽量避免在合约代码中编写复杂的计算逻辑和存储大数据。不同的区块链有不同的限制设定，一般资源限制设定包含：运行超时限制；最大可使用内存限制；系统底层资源访问限制；受限的外部网络访问。
- 2. 无状态原则：不宜在智能合约内部使用全局变量，建议设计为无状态(stateless)。状态代表存储的信息，无状态即一次操作，不能保存数据。智能合约处理的状态多数存在于账本中，而不是存在于智能合约自身结构中。因此，智能合约中的方法最好都设计成“函数式”风格，不依赖临时变量等方式记录之前的运行结果。若在某些场景下需要支持有状态服务，如出错时进行重试等，有状态的逻辑代码应该放在上层的应用中实现。
- 3. 确定性原则：当多个相同智能合约的实例对同一调用得到彼此不相同的结果时，网络无法达成共识此时智能合约的调用运行实际上将无法完成。因此，智能合约中的方法需保证不会出现非确定性的逻辑。若在某些场景下需要支持非确定性设计逻辑，如抽奖需要采用随机数生成等，非确定性逻辑实现应避免放在智能合约中，而是放在上层应用通过代理服务等方式完成。

智能合约开发步骤

■ 步骤二：开发智能合约

■ 不同区块链系统的智能合约开发方法

- 不同区块链系统的智能合约实现方式不同。在开发智能合约方法过程中，应提前清楚地理解其对应区块链系统的运行机制，了解智能合约的结构。注意有的区块链系统会支持开发模式和日志模块。尽量使用开发模式开启智能合约开发流程，因为在开发模式下可以自由地修改代码而无须重新部署智能合约，也无须多次重启区块链系统网络。
- 使用日志模块是一项有用的开发技巧，能够帮助调试智能合约并快速找出智能合约的缺陷。开发智能合约时应阅读对应区块链系统的参考文档，理解其区块链系统运行机制和智能合约结构，并了解如何使用开发模式和日志模块。

■ 智能合约的调用与查询

- 调用操作会执行智能合约并会把执行结果记录下来生成事务，最终生成区块并同步到所有区块链节点中。而查询操作只会读取区块链系统账本中的状态，不生成事务，也不会将执行结果写入区块链。
- 开发智能合约时需考虑执行智能合约时是否涉及状态修改。若有状态修改应使用调用操作，若只需要查询而不修改状态，应使用查询操作。

智能合约开发步骤

■ 步骤二：开发智能合约

■ 智能合约的异常处理

- 通过异常处理，可以对用户在智能合约中的非法输入进行控制和提示，以防智能合约崩溃，还能用于通知外界该智能合约不能正常执行，如输入的数据无效(例如除数是0)，或所需资源不可用(例如文件丢失)等。
- 智能合约的异常处理方式通常采用终止模型。在终止模型中如果错误非常关键，将导致智能合约无法返回到异常发生的地方继续执行。抛出异常后会终止并回滚目前执行的调用，然后将错误信息通知用户。

■ 智能合约事件

- 事件是从合约方法中发出的通知，以表明在合约方法执行期间是否存在某个条件或者标志。事件提供了函数返回值以外的功能，便于区块链通知应用程序，是区块链系统日志功能中提供的一组接口。
- 当事件被触发时，会将事件参数保存到交易日志中。同时事件可以被监听，用户可以通过订阅事件的方式及时获取相应事件日志记录信息。智能合约开发时可以考虑在某个动作发生前后是否需要触发事件。如进行转账操作时，可通过事件方式通知用户转账是否成功。

智能合约开发步骤

■ 步骤三：部署和测试智能合约

- 完成合约方法编码后，需要通过区块链客户端或者在线集成开发环境(IDE)等工具来实现对智能合约的部署和测试。智能合约的开发类似于硬件开发，是不变的代码。一旦部署后，它们将是无法更新的最终版。因此在部署前，必须对智能合约进行彻底的测试。
- 在测试智能合约时应进行单元测试。单元是一个智能合约中最小可测试部分，在单元测试活动中，智能合约的独立函数方法将在与智能合约的其他部分相隔离的情况下进行测试。通过单元测试来确保所编写的智能合约代码匹配项目需求和遵循开发目标。单元测试可以手动测试，也可以自动测试，通常采用自动测试。
- 单元测试旨在推动智能合约的开发。单元测试代码的结果决定了能否检测出智能合约存在的隐藏漏洞。由于不同的区块链系统采用不同的方法实现，测试实现方式也有所不同，但一般支持以下两种方式编写简单且易于管理的测试。
 - (1)编写测试脚本，从客户端中执行智能合约，进行测试。
 - (2)编写测试合约，从其他智能合约调用被测智能合约，进行测试。
- 同时，健壮的单元测试过程应包括两种不同类型的测试：
 - (1)正面测试(positive testing):测试智能合约是否能完成它应该完成的工作。确保智能合约中的每个方法能在提供有效输入集的情况下正确执行并按预期执行。
 - (2)负面测试(negative testing):测试智能合约是否不执行它不应该完成的操作。确保智能合约在验证和确认过程中能捕获到错误，并且在给出非法输入时会终止并撤回操作。

智能合约案例-交易转账[FABRIC]

■ 案例简介

- 超级账本Fabric 项目中自带一些链码(chaincode), 即智能合约的示例, 本案例是用 Go语言编写的一个实现账户A和 B之间相互转账功能的链码。代码来源于超级账本 Fabric 1.4 版本的 `examples/chaincode/go/chaincode _example02/chaincode -exam-ple02.go`。
- 在该链码中, 采用无状态设计原则, 先从区块链系统中获取双方余额, 之后计算转账后的余额, 再将双方余额写入区块链。为了简化流程便于理解, 转账操作时对用户余额未做严格要求, 允许用户余额为负, 但在实际工作中, 需要做相应的检查。
- 在超级账本Fabric 中, 智能合约是通过链码来实现的。链码分为系统链码和用户链码。系统链码提供系统层面的功能, 用户链码提供用户的上层应用功能。一般所说的链码是指用户链码。Fabric 支持多种编程语言编写链码, 可采用 Go、Java、Nodejs、JavaScript 等编写。在开发的过程中, 开发者需要编译和执行链码。
- 本案例选择 Go语言作为链码的开发语言, 选择 Visual Studio Code(简称 VS Code)作为链码的集成开发环境。VSCode可使用IBM Blockchain Platform 的插件来简化开发、测试和部署超级账本Fabric链码的过程。

智能合约案例-交易转账[FABRIC]

■ 案例设计

- 整个案例的运行逻辑分为初始化用户、用户转账、查询余额、删除用户四部分。
- 本案例的链码实现了调用(invoke)的三个子方法。Invoke被调用时会根据交易传入的参数定位到不同的子方法处理逻辑。

表 1-1 转账合约方法总结

方法	子方法	功能	说明
Init	无	初始化用户	添加两个用户并初始化用户的余额
Invoke	invoke	用户转账	一个用户向另一个用户转账
	query	查询余额	查询一个用户的余额
	delete	删除用户	删除一个用户

根据案例的运行逻辑可设计出以下转账合约类图，如图 1-1 所示。

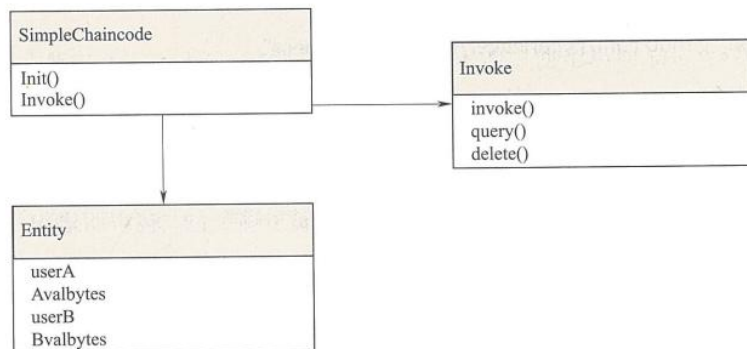


图 1-1 转账合约类图

智能合约案例-交易转账[FABRIC]

■ 案例开发

- 根据案例的合约类图，按照链码的结构先声明实体结构再依次实现功能函数方法。
- 实体结构声明 接口实现

引用相关包和声明转账合约中涉及的实体结构。

```
package example02

import (    // 引用链码所需要的相关包
    "fmt"
    "strconv"

    "github.com/hyperledger/fabric/core/chaincode/shim"
    pb "github.com/hyperledger/fabric/protos/peer"
)

// 声明结构体并实现相应接口
type SimpleChaincode struct {
```

根据链码结构需要实现 Init 和 Invoke 接口。Init 方法设定用户 A 和 B 的初始余额。
Invoke 方法通过调用子方法实现相应用户转账、删除用户和查询余额等功能。

```
// 初始化用户,实现 Init 接口
func (t * SimpleChaincode) Init(stub shim.ChaincodeStubInterface) pb.Response {
    fmt.Println("ex02 Init")           // 打印信息,表明调用 Init 方法
    _ , args := stub.GetFunctionAndParameters() // 获取交易传入参数
    var A, B string                     // 用户
    var Aval, Bval int                 // 用户余额
    var err error
    if len(args) != 4                  // 判断交易传入参数个数
        return shim.Error("Incorrect number of arguments. Expecting 4")
    }
    A = args[0]                       // 初始化用户 A
    Aval, err = strconv.Atoi(args[1]) // 字符串转整数,初始化用户 A 余额
    if err != nil {
        return shim.Error("Expecting integer value for asset holding")
    }
}
```

```
B = args[2]
Bval, err = strconv.Atoi(args[3])
if err != nil {
    return shim.Error("Expecting integer value for asset holding")
}
fmt.Printf("Aval = % d, Bval = % d\n", Aval, Bval) // 打印用户 A 和 B 的余额
// 将用户余额写入区块链
err = stub.PutState(A, []byte(strconv.Itoa(Aval))) // 整数转字符串,将用户 A
余额写入区块链
if err != nil {
    return shim.Error(err.Error())
}
} else if function == "query" {
    return t.query(stub, args) // 查询余额
}
// 若调用的子方法名称不对则返回错误信息
return shim.Error("Invalid invoke function name. Expecting \"invoke\" \"delete\" \"
query\"")
}

return shim.Success(nil)
}

// 实现 Invoke 接口
func (t * SimpleChaincode) Invoke(stub shim.ChaincodeStubInterface) pb.Response {
    fmt.Println("ex02 Invoke")           // 打印信息,表明调用 Invoke 方法
    function, args := stub.GetFunctionAndParameters() // 获取交易传入参数
    if function == "invoke" {             // 判断传入的子方法名称是否为"invoke"
        return t.invoke(stub, args)       // 用户转账
    } else if function == "delete" {
        return t.delete(stub, args)       // 删除用户
    }
}
```

智能合约案例-交易转账[FABRIC]

■ 案例开发

- 子方法实现：在子方法中具体实现用户转账、删除用户和查询余额等功能。

1) 用户转账

用户转账需传入三个参数：转账人名称、被转账人名称、转账金额。

用户转账按照三步流程实现：从区块链中获取用户余额、用户余额转账计算、用户余额写入区块链。若操作过程中出现错误则直接终止操作并返回错误信息。

// 用户转账实现

```
func (t * SimpleChaincode) invoke(stub shim.ChaincodeStubInterface, args []string) pb.
```

```
Response {
```

```
    var A, B string          // 用户
```

```
    var Aval, Bval int       // 用户余额
```

```
    var X int                // 转账金额
```

```
    var err error
```

```
    if len(args) != 3 {      // 判断交易传入参数个数
```

```
        return shim.Error("Incorrect number of arguments. Expecting 3")
```

```
    }
```

```
    A = args[0]
```

```
    B = args[1]
```

```
    // 从区块链中获取用户余额
```

```
    Avalbytes, err := stub.GetState(A)
```

```
    if err != nil {
```

```
        return shim.Error("Failed to get state")
```

```
    }
```

```
    if Avalbytes == nil {      // 用户余额不存在则返回错误信息
```

```
        return shim.Error("Entity not found")
```

```
    }
```

```
    Aval, _ = strconv.Atoi(string(Avalbytes))
```

```
    Bvalbytes, err := stub.GetState(B)
```

```
    if err != nil {
```

```
        return shim.Error("Failed to get state")
```

```
    }
```

```
    if Bvalbytes == nil {
```

```
        return shim.Error("Entity not found")
```

```
    }
```

```
    Bval, _ = strconv.Atoi(string(Bvalbytes))
```

```
    X, err = strconv.Atoi(args[2]) // 字符串转整数，X 为转账金额
```

```
    if err != nil {
```

```
        return shim.Error("Invalid transaction amount, expecting a integer value")
```

```
    }
```

```
    Aval = Aval - X // 转账操作
```

```
    Bval = Bval + X
```

```
    fmt.Printf("Aval = %d, Bval = %d\n", Aval, Bval) // 打印转账后用户余额
```

```
    // 将转账后的用户余额写入区块链
```

```
    err = stub.PutState(A, []byte(strconv.Itoa(Aval)))
```

```
    if err != nil {
```

智能合约案例-交易转账[FABRIC]

■ 案例开发

- 子方法实现：在子方法中具体实现用户转账、删除用户和查询余额等功能。

```
return shim.Error(err.Error())
}
```

```
err = stub.PutState(B, []byte(strconv.Itoa(Bval)))
```

```
if err != nil {
    return shim.Error(err.Error())
}
```

```
return shim.Success(nil)
```

```
}
```

2) 删除用户

删除用户操作只需要传入需要删除的用户名称。若删除用户失败则需要异常处理，执行一个返回错误信息的操作。

// 删除用户实现

```
func (t * SimpleChaincode) delete(stub shim.ChaincodeStubInterface, args []string) pb.
```

```
Response {
    if len(args) != 1 { // 判断交易传入参数个数
        return shim.Error("Incorrect number of arguments. Expecting 1")
    }
    A := args[0] // 传入想要删除用户的名称
```

// 从区块链中删除用户

```
err := stub.DelState(A)
if err != nil {
    return shim.Error("Failed to delete state")
}
```

```
return shim.Success(nil)
}
```

3) 查询余额

查询用户余额操作只需要传入需要查询的用户名称。若查询用户余额失败则需要执行一个返回错误信息的操作。

// 查询余额实现

```
func (t * SimpleChaincode) query(stub shim.ChaincodeStubInterface, args []string) pb.
```

```
Response {
    var A string // 用户
    var err error

    if len(args) != 1 { // 判断交易传入参数个数
        return shim.Error("Incorrect number of arguments. Expecting name of the person to
        query")
    }
    A = args[0] // 传入想要查询余额的用户名称
```

// 从区块链中查询用户余额

```
Avalbytes, err := stub.GetState(A)
if err != nil {
    jsonResp := "{\"Error\":\"Failed to get state for " + A + "\"}"
    return shim.Error(jsonResp)
}
```

if Avalbytes == nil { // 若用户余额不存在则返回错误信息

```
jsonResp := "{\"Error\":\"Nil amount for " + A + "\"}"
return shim.Error(jsonResp)
```

```
}
jsonResp := "{\"Name\":\"" + A + "\",\"Amount\":\"" + string(Avalbytes) + "\"}"
fmt.Printf("Query Response:% s\n", jsonResp) // 打印用户余额
return shim.Success(Avalbytes)
```

```
}
```


智能合约案例—交易转账[FABRIC]

■ 案例测试

- 由于单元测试运行过程中并未与真实区块链系统进行交互，MockInit()和MockInvoke()函数中 uuid 参数值在测试中可任意设置，在此示例中 uuid 值设置为1，合约对象取名为 simplecc。

新建单元测试文件 chaincode_test.go。根据所测试链码编写测试用例。下面是 chaincode_example02.go 转账链码的一个单元测试用例。

```
package example02          // 包名与链码的包名需一致

import (
    "fmt"
    "github.com/hyperledger/fabric/core/chaincode/shim"
    "testing"                // 需要引入 testing 包
)
```

```
func TestTx (t * testing.T) {    // 测试函数以 Test 开头，后缀首字母大写
```

```
    cc := new(SimpleChaincode)    // 创建链码对象
    stub := shim.NewMockStub("simplecc", cc)    // 创建 MockStub 对象
```

// 调用 Init 接口，初始化用户 A 余额为 100，用户 B 余额为 50

```
stub.MockInit("1", [][]byte{[]byte("A"), []byte("100"), []byte("B"), []byte("50")})
```

// 调用 query 子方法，查询用户 A 余额

```
res := stub.MockInvoke("1", [][]byte{[]byte("query"), []byte("A")})
```

```
fmt.Println("The balance of A is ", string(res.Payload))
```

// 调用 invoke 子方法，令用户 A 向用户 B 转账 50

```
res := stub.MockInvoke("1", [][]byte{[]byte("invoke"), []byte("A"), []byte("B"), []byte("50")})
```

// 再次调用 query 子方法，查询用户 A 余额

```
res := stub.MockInvoke("1", [][]byte{[]byte("query"), []byte("A")})
fmt.Println("The new balance of A is ", string(res.Payload))
}
```

该测试用例实现了用户 A 与 B 之间转账的正面测试，并在过程中打印用户 A 的余额情况。

将单元测试文件 chaincode_test.go 放至 chaincode_example02.go 转账链码同一目录下，并执行 go test 命令，即可得到如图 1-2 所示的测试结果。

```
root@ubuntu:~/usr/lib/go/src/example02# go test
The balance of A is 100
The new balance of A is 50
PASS
ok      example02      0.006s
root@ubuntu:~/usr/lib/go/src/example02#
```

图 1-2 正面测试结果

同时对应地编写一个负面测试用例，新建单元测试文件 chaincodeInvalid_test.go。在该测试用例中模拟用户转账金额数值非法，并在过程中打印执行结果情况。

```
package example02          // 包名与链码的包名需一致

import (
    "fmt"
    "github.com/hyperledger/fabric/core/chaincode/shim"
    "testing"                // 需要引入 testing 包
)
```

```
func TestInvalidTx(t * testing.T) {    // 负面测试
```

```
    cc := new(SimpleChaincode)    // 创建链码对象
```

```
    stub := shim.NewMockStub("simplecc", cc)    // 创建 MockStub 对象
```

// 调用 Init 接口，初始化用户 A 余额为 100，用户 B 余额为 50

```
stub.MockInit("1", [][]byte{[]byte("A"), []byte("100"), []byte("B"), []byte("50")})
```

// 调用 invoke 子方法，令用户 A 向用户 B 转账，转账金额为非法的数字

```
res := stub.MockInvoke("1", [][]byte{[]byte("invoke"), []byte("A"), []byte("B"), []byte("InvalidNumber")})
```

// 打印调用返回结果信息中的状态与内容

```
fmt.Println("response Status is ", string(res.Status))
```

```
fmt.Println("response Body is ", string(res.Payload))
}
```

将单元测试文件 chaincodeInvalid_test.go 放至 chaincode_example02.go 转账链码同一目录下，并执行 go test 命令，即可得到如图 1-3 所示测试结果。

```
root@ubuntu:~/usr/lib/go/src/example02# go test
response Status is 500
response Body is Invalid transaction amount, expecting a integer value
PASS
ok      example02      0.005s
root@ubuntu:~/usr/lib/go/src/example02#
```

图 1-3 负面测试结果

智能合约案例-投票[ETH]

■ 案例简介

- 本案例是使用Solidity 语言实现的一个自动化且透明的投票智能合约。代码来源于“Solidity 官方文档”中的示例。在该投票合约中: (1)投票发起人可以发起投票, 将投票权赋予投票人。(2)授权过后的投票人可以投票, 但不允许重复投票。(3)任何人都可以查询投票的结果。
- 以太坊智能合约的运行, 和Fabric的步骤一致, 包括编码、编译、部署、运行。以太坊使用图灵完备的高级编程语言solidity开发智能合约, solidity运行在以太坊虚拟机 (EVM) 上。通过EVM编译器编译为字节码, 最终通过客户端部署到以太坊网络中。
- 本案例使用Remix开发智能合约, Remix是基于浏览器使用solidity语言的集成开发环境。

Remix 由功能面板、侧面板、主面板、终端四部分组成, 如图 1-4 所示。

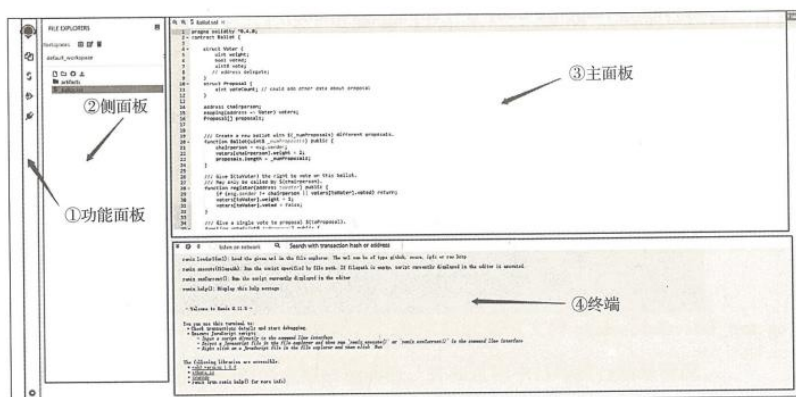


图 1-4 Remix 布局

表 1-2

Remix 布局各模块说明

模块	说明
功能面板	点击以切换哪个选项页在侧面板显示, 从上至下依次是文件浏览、编译、运行、插件管理
侧面板	展示选项页的操作界面。在“文件浏览”选项页中显示已编写的智能合约相关目录与文件
主面板	主要用于编辑文件。根据选项页的不同, 会显示用于集成开发环境 (IDE) 编译的插件或文件的具体内容
终端	在该部分显示合约执行或静态分析的运行结果, 同时也可以运行脚本进行交互

智能合约案例-投票[ETH]

■ 案例设计

- 整个案例的运行流程主要分为发起投票、用户授权、用户投票、查询投票结果四部分。
- 合约类图如图1-6所示，投票合约中涉及的实体有主席、投票人、提案。投票合约由发起投票、用户授权、用户投票、查询投票结果四个方法组成。

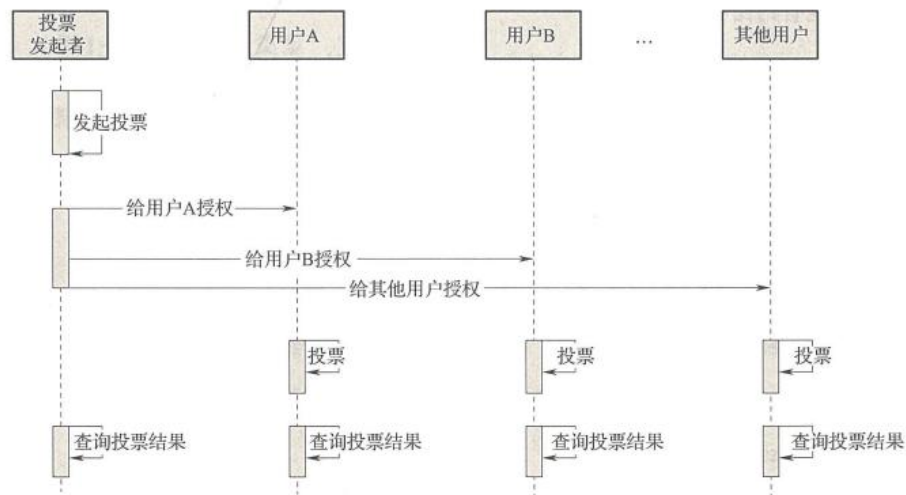


图 1-5 投票合约时序图

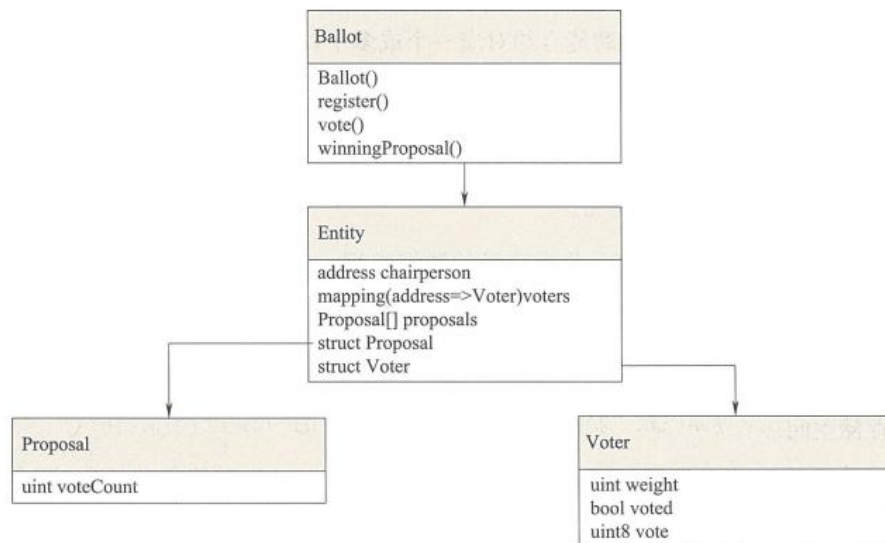


图 1-6 投票合约类图

智能合约案例-投票[ETH]

■ 案例开发

- 根据案例的合约类图，按照智能合约结构先声明实体结构再依次实现功能函数方法。

(1) 声明实体结构

按照合约类图声明投票合约中涉及的所有实体结构。

```
struct Voter {           // 声明投票人结构体
    uint weight;          // 权重
    bool voted;           // 是否已投票
    uint8 vote;           // 投票给哪个提案
}

struct Proposal {        // 声明提案结构体
    uint voteCount;       // 票数
}

address chairperson;     // 主席
mapping(address => Voter) voters; // 投票人
Proposal[] proposals;    // 提案数组
```

(2) 发起投票

投票发起人充当主席 chairperson 的身份，负责发起投票。在此案例中通过创建新的投票合约的方式来表示发起投票。

```
// 发起一个指定数量提案的投票
function Ballot(uint8 _numProposals) public {
    chairperson = msg.sender; // 指定主席
    voters[chairperson].weight = 2; // 设置主席投票权重为 2
    proposals.length = _numProposals; // 设置提案数量
}
```

(3) 用户授权

投票发起人给投票人授权。

```
// 用户授权
function register(address toVoter) public {
    // 只有主席能调用且用户只能被授权一次
    if (msg.sender != chairperson || voters[toVoter].voted) return;
    voters[toVoter].weight = 1; // 用户投票权重为 1
    voters[toVoter].voted = false; // 表明用户尚未投票
}
```

(4) 用户投票

用户对投票提案进行投票。只有已经被授权的用户才能参与这次投票并且不允许 // 用户投票

```
function vote(uint8 toProposal) public {
    Voter storage sender = voters[msg.sender]; // 获取函数调用方用户身份
    if (sender.voted || toProposal >= proposals.length) return; // 用户只能投一次
    // 用户投票
    // 票且只能给存在的提案投票
    sender.voted = true; // 表明用户已投票
    sender.vote = toProposal; // 记录用户投票给哪个提案
    proposals[toProposal].voteCount += sender.weight; // 提案投票数计数
}
```

(5) 查询投票结果

投票结束后，智能合约自动计票并且所有用户均可查询这次投票的结果。

```
// 查询投票获胜结果
function winningProposal() public constant returns (uint8 _winningProposal) {
    uint256 winningVoteCount = 0;
    for (uint8 prop = 0; prop < proposals.length; prop++) // 循环遍历提案
        // 票数
        if (proposals[prop].voteCount > winningVoteCount) {
            winningVoteCount = proposals[prop].voteCount; // 记录票数最高
            // 提案
            _winningProposal = prop;
        }
}
```

区块链应用系统开发案例

■ 案例背景介绍

- 经典电商平台模型中，用户因某商户对外的广告宣传而注册成为平台用户后，随后却与其他商户发生交易。对于投入广告宣传的商户来说，广告宣传不仅为平台赢得流量，也为其他商户导入了流量，但是自己却没有取得相应的回报。区块链电商联盟平台的概念因此诞生。
- 由几家商户共同发起基于区块链联盟平台的建设，在联盟平台内，有两种角色:商户和用户。假设一个场景:A商户通过广告宣传等方式，获得一个用户并在平台注册账号，该用户对于平台来说，是由A商户引流获得，若该用户与平台内其他商家成功发生交易后，为保证 A商户前期投入的权益，B 商户将按照约定的规则，给予A商户一定的导流费。电商联盟平台模型如图2-4如示。

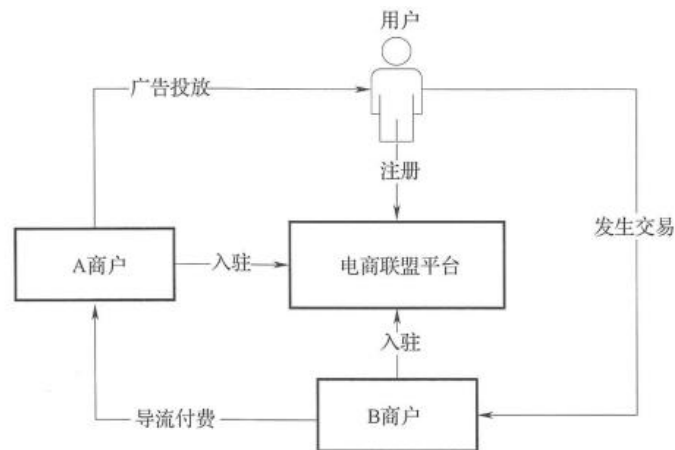


图 2-4 电商联盟平台模型

区块链应用系统开发案例

■ 实现人机交互界面

- 注册与登录；商品与查询；订单与交易；导流与结算
- 订单详情中，商品只需展示名称型号或简介即可，点击商品可查看商品详情快照。在订单信息中，需要展示编号、收货信息、创建和付款发货等时间，需要留有各个功能入口，包括物流详情的入口、商品售后的入口和评价等。
- 导流与结算：商家需要页面查看自己导流的汇总信息，查看每月结算，并且可以通过日期等查询字段快速筛查，也可查询每月详细信息。为了保证用户的隐私，不可展示对具体到某个用户的导流信息，应以商家对商家的形式展现。



图 2-5 用户注册登录参考实现



图 2-6 应用页面参考实现



图 2-7 订单详情参考界面



图 2-8 导流信息页面参考实现

区块链应用系统开发案例

■ 智能合约开发

- 整个案例的运行逻辑为初始化合约，可分为商户入驻、用户注册、订单交易、查询订单、查询余额五部分。
- 本案例的链码实现了 Invoke 的五个子方法: addMerchant(商户入驻)、registerUser(用户注册)、newOrder(订单交易)、findOrder(查询订单)、queryToken(查询余额)。Invoke 被调用时会根据交易传入的参数定位到不同的子方法处理逻辑。

表 2-6 电商平台合约方法总结

方法	子方法	功能	说明
Init	无	初始化合约	初始化合约，不需要做额外操作
Invoke	addMerchant	商户入驻	新增商户信息并初始化余额
	registerUser	用户注册	新增用户，绑定引流的商户
	newOrder	订单交易	订单存储上链，商户之间根据约定好的结算规则缴纳导流费
	findOrder	查询订单	根据订单号查询订单
	queryToken	查询余额	查询商户余额

根据案例的运行逻辑可设计出电商平台合约类图，如图 2-9 所示。

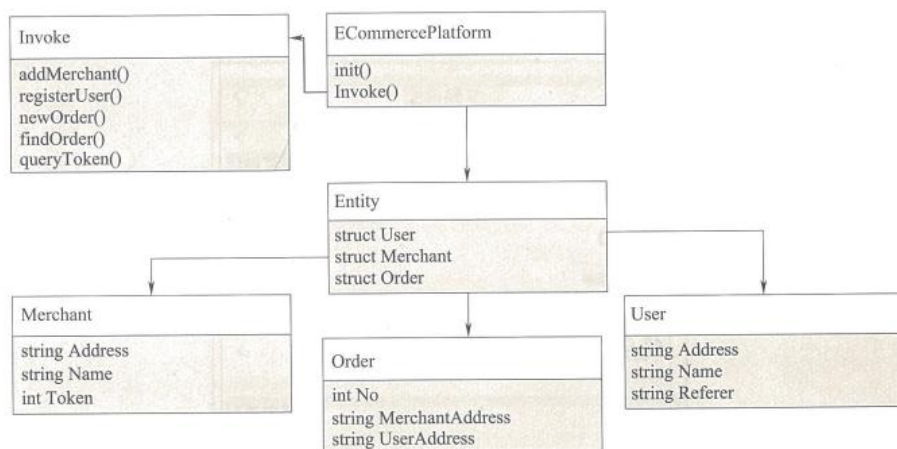


图 2-9 电商平台类图

区块链应用系统开发案例

■ 开发后端服务

- 后端开发内容部分可采用Java实现
- 区块链访问层实现：区块链访问层实现调用软件开发工具包与智能合约交互。根据智能合约调用方法定义相对应的接口。
- 数据访问层实现：数据访问层的接口定义需要访问数据库的相关对象操作。在本案例中，商户、用户和订单等数据对象都需要访问数据库获得。
- 业务逻辑层实现
 - 商户入驻：新商户入驻时需要给商户生成公私钥，公钥当作商户的地址，在生产环境下私钥需加密存储。
 - 新用户注册：在实现注册接口时，需要定义接收的参数，包括账户密码、推荐人等，并需要查询是否已经存在，避免重复注册。验证通过后，通过区块链软件开发工具包调用合约的注册方法。调用合约时，需要注意正确传递合约地址或ID相关参数，若合约调用失败，则需要终止注册流程并返回注册失败信息。
 - 订单交易：用户发起创建订单请求时，先要通过商品ID获取商品的详细信息，判断是否有货，获取下单时的价格(是否引入优惠活动)，对商品详情做一个快照，存入该笔订单中。校验参数时，为了避免向本地数据库存入脏数据，导致链上数据与本地数据不同步，需要优先调用软件开发工具包将数据上链，成功返回后再将数据持久化至本地数据库。
 - 实现web服务接口：注册；交易；查询等实现。

区块链应用系统开发案例

■ 开发经验总结

- 撰写清晰、完整、准确的设计文档并及时更新
 - 在开发多人合作的应用系统时，为了提高工作效率并保证工作质量，需要制定一份完善的设计文档并利用协作平台及时更新，保证所有开发人员及时获得最新信息。各组开发人员可以按照设计文档并行开发，节约开发时间。
- 在代码层面优化系统
 - 对于电商平台，高并发的访问量是一定会面对的问题，需要考虑如何使用代码，使有限的硬件得到充分的利用。可以从以下方面优化:优化线程、优化网络请求、控制静态文件大小。
- 合理选择上链数据
 - 应用系统与底层区块链调试时，应尽量避免将无用或错误的数据上链。由于区块链的特性，这些无意义的数字将会永远被保存下来。在系统联调时要重复测试所有可能发生的情况。